

# Introduction to Machine Learning

## Video Lesson

Duration: 00:24:43

## Slide Deck

**M12S01** (<https://canvas.tamu.edu/courses/430127/files/92570694?wrap=1>) 

## Lesson Transcript

This lecture is the first lecture in the machine learning module. The content of this lecture will be a basic introduction to machine learning.

---

# What is Machine Learning?

- Improve performance in a task through experience.
  - From rote memory, all the way up to learning a complex strategy
- Types of machine learning:
  - ① Supervised learning (dense feedback)
  - ② Unsupervised learning (no feedback)
  - ③ Reinforcement learning (sparse feedback, environment altering), etc.
- Advantages:
  - ① Fault tolerance
  - ② No need for a complete specification to begin with
  - ③ Learning from data
  - ④ Superhuman performance in some cases
- Has become a dominant force in AI since the mid 1990's.

First, let us ask this question, what is machine learning? There is a very brief summary that tells us what machine learning is. In one sentence, we can say any algorithm that improves performance in a task through some experience.

The scope of machine learning is very broad. It can include rote memory all the way up to learning a very complex strategy. There are three basic types of machine learning, supervised learning, unsupervised learning, and reinforcement learning. We will get to more details about these three types later on, but first, let me mention that in supervised learning, given any input data point, we get pretty dense feedback. In unsupervised learning, on the other hand, we only have the input data points, and we need to figure out something about it, so there is really no feedback or teacher signal associated with each input. Reinforcement learning is very different from these two. In this case, we get occasional feedback. So, that is past feedback based on our performance, and as a result of the action of the agent that performs reinforcement learning, the environment can change.

Here are some advantages of machine learning. It is fault-tolerant. There is no need for a complete specification to begin with, and we can learn from data, and sometimes we can achieve superhuman performance. Machine learning has become a dominant force in AI since the mid-1990s.

---

# Why Machine Learning?

- Abundance of data: the data deluge.
  - Scientific instruments.
  - Data acquisition devices.
  - Internet and the web.
  - All sectors of human society producing and digitizing data (e.g., your cell phone).
- Not enough human expertise or human power to make sense of such huge amounts of data.

Why did machine learning emerge as a dominant player in the field of AI? It is basically due to these two things. First, the abundance of data, and second, not enough human expertise or human power to make sense of such huge amounts of data.

Several different data sources add to this abundance of data. Lots of scientific instruments. We can think about telescopes, microscopes, and all sorts of scientific instruments measuring different aspects of the natural world. And associated with that, we have a lot of data acquisition devices.

Furthermore, since the early 1990s, the internet and the web have accumulated huge amounts of data. And all sectors of human society are producing and digitizing data. We can consider our cell phones as huge data collection devices as well. It has a microphone; it has a video camera, it has GPS sensors, and so on.

---

## Machine Learning in the News



IBM's Watson



Google DeepMind's AlphaGo

- IBM's Watson beats human champions: Jeopardy (2011) <sup>1</sup>
- Google's Artificial brain learns to find cat videos in YouTube (2012).
- Google DeepMind: Atari 2600 game playing, AlphaGo, AlphaStar (2015~)
- OpenAI: ChatGPT (2022 ~)

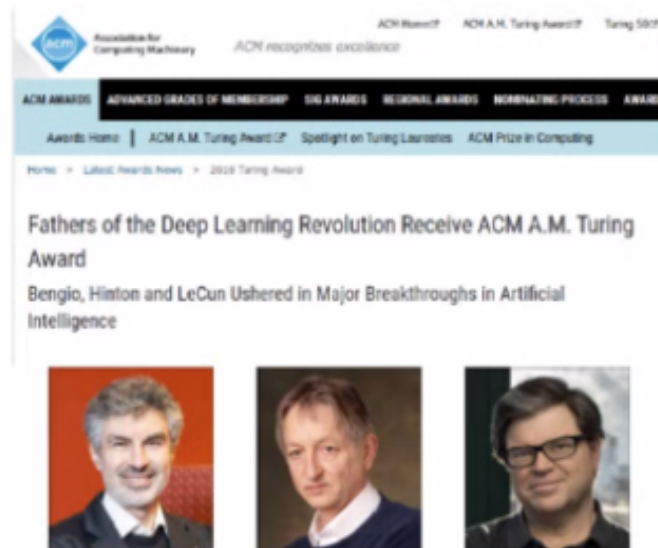
Machine learning has been in the news a lot, especially very recently. Well, it started maybe around 2011 - 2012. This is when IBM's Watson Machine Learning System beat human champions in the game show Jeopardy. We can see a snapshot of that event here.

In 2012, Google's artificial brain algorithm launched to find cat videos on YouTube. So, this was already more than 10 years ago. And yes, nowadays, deep learning algorithms can do something like this routinely. But back then, 10 years ago, or 10 or more years ago, this was a huge deal.

In 2015, Google DeepMind started making real progress in deep reinforcement learning and approaching human-level performance in pretty difficult tasks like playing video games like the Atari 2,600 game or beating a human expert or human master in the game Go in 2016. This was a huge achievement at the time. Then, they extended their approach to play more complex games, such as real-time strategy games like StarCraft.

And as we know, starting last year, there has been a huge success in large language models like ChatGPT.

## ACM Turing Award 2018: Deep Learning



- ACM Turing Award 2018 goes to Deep Learning pioneers!

Deep learning is the crown paradigm in machine learning. This revolution started around 2010 - 2012 and culminated in these three pioneers in deep learning being awarded the ACM Turing Award.

This was for 2018. Yoshua Bengio, Geoffrey Hinton, and Yann LeCun pioneered deep learning approaches.

---

# What Does It Take to do ML?

A lot of math:

- Linear algebra
- Calculus
- Probability and statistics
- Differential geometry
- Numerical methods

So, what does one need to know? What are the basic skills that we need to be successful machine learning researchers or machine learning engineers? We need a lot of math, like linear algebra, calculus, probability, and statistics. If we want to go into more advanced stuff, we need to know differential geometry.

If we want to write numerical simulations to simulate these neural networks, we also need to know about numerical methods.

---

# Types of Machine Learning

## ① Supervised learning

- Input-Target pairs
- $\{\langle \vec{x}_i, \vec{t}_i \rangle | i = 1, 2, \dots, n\}$

## ② Unsupervised learning

- A bunch of inputs (unlabeled)
- $\{\vec{x}_i | i = 1, 2, \dots, n\}$

## ③ Reinforcement learning

- $\text{state}_1 \xrightarrow{\text{action}_1} \text{state}_2 \xrightarrow{\text{action}_2} \text{state}_3, \dots, \text{reward}$
- $s_{t+1} = \delta(s_t, a_t), r_{t+1} = \rho(s_t, a_t)$

Let us revisit the three types of machine learning in more detail.

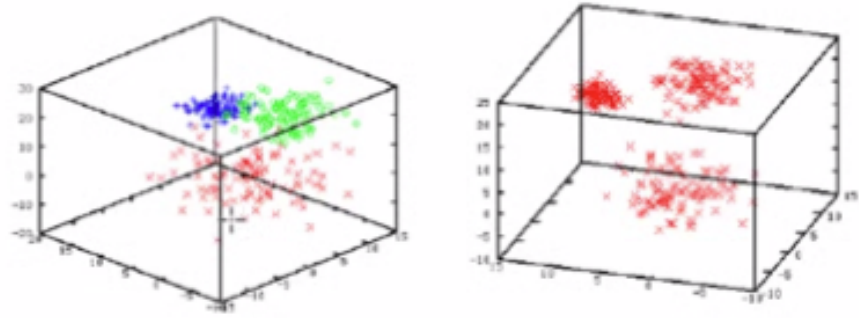
First is supervised learning. Here, we are given a pair of input and target values and a huge collection of those. The task is given a new value,  $x$ , that is not in the training set, and we want to predict what the correct target value is.

In unsupervised learning, we just get a bunch of inputs, and based on that, we want to discover some features or structures in the data set. So, there is no teacher signal here or a target signal.

In reinforcement learning, the setting is very different. We start in a state. Based on that state, we take action, and we transition into the next state, and as a result, we get some kind of reward. The goal is to maximize the reward that we collect as we take the sequence of actions.

---

# Example Data



- Left: supervised (three classes: red, green, blue)
- Right: unsupervised (no class info)
- Typically very high dimensional (10,000, 1 million [or more]).

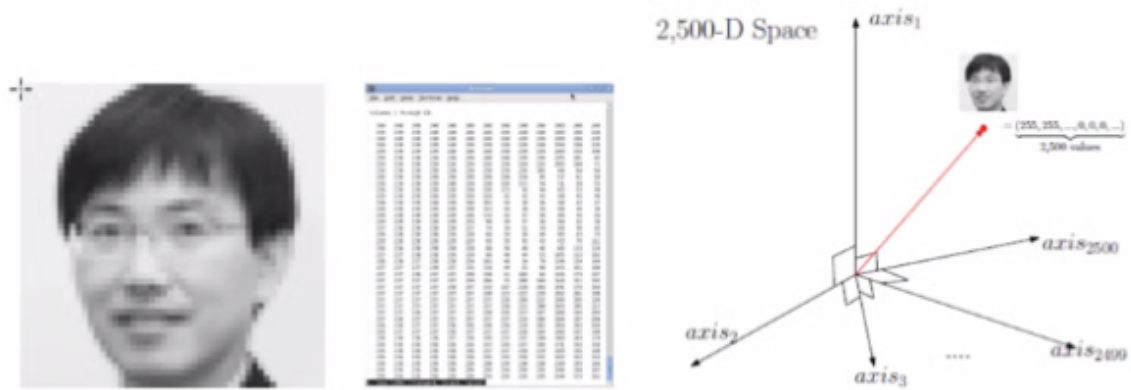
Here we can see two data sets. On the left-hand side is a typical supervised dataset, and on the right-hand side is a typical unsupervised dataset. In a supervised dataset, for each data point, there is a color that represents the three different classes, red, green, and blue. Whereas for the unsupervised dataset, there is no color difference between these three clouds of data points.

The task is to find out some structure. For example, there is one cluster of data points there that may belong to the same type or something. Here all we can say is that there are three groups of data points.

These samples are from a 3-dimensional space. The input dimensionality for this dataset is 3, whereas typically, the dimensionality can be very high, like 10,000 or even 1 million.

---

# High-dimensional Data



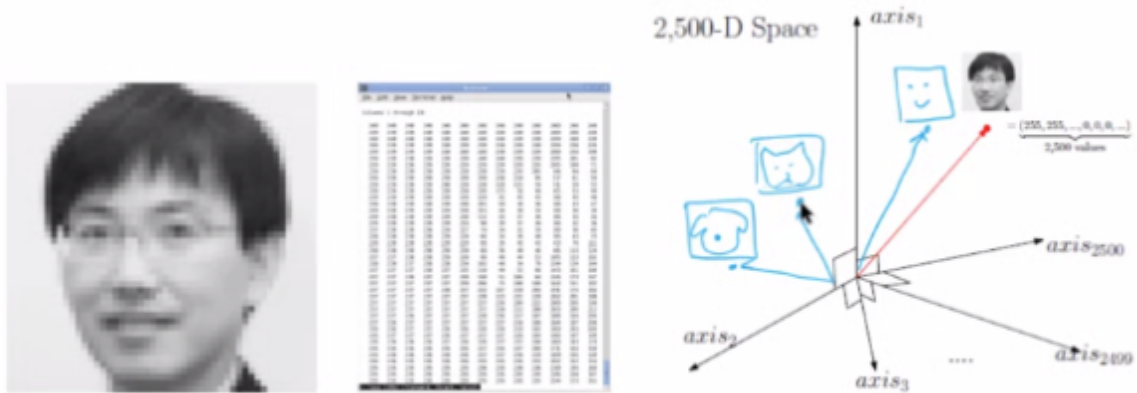
- Images: these are images on a 2D plane, but ...
- These are, data-wise,  $50 \times 50 = 2,500$ -dimensional vectors.
  - Each such image is a single point in 2,500-dimensional space.

Consider a collection of images like that. It is grayscale, and let us say it is 50 by 50. When we write down the pixel value of this image into a matrix like that and subsequently turn it into a vector, then we have 50 by 50 values corresponding to each pixel. For these light gray, we will have a value like 240, where 0 means totally black and 255 means totally white. So we have a 50 by 50 vector. So, for this single image, if we plot it in a 2,500-dimensional space, here are axis 1, and axis 2, all the way up to axis 2,500.

Then this single point in this 2,500-dimensional space corresponds to this image of myself.

---

# High-dimensional Data



- Images: these are images on a 2D plane, but ...
- These are, data-wise,  $50 \times 50 = 2,500$ -dimensional vectors.
  - Each such image is a single point in 2,500-dimensional space.

If we have some different images, let us say some other face that looks similar to me, then this point, this 2,500-dimensional point, would be near my vector here. If we have different images that look different from my face, for example, a dog or a cat, they might also be a single point in this 2,500-dimensional space, but they will be located a bit far away from my data point here.

---

## Supervised Learning

- Regression: approximating  $y = f(x)$
- Classification: face recognition, hand-written character recognition, credit risk assessment, etc.
- Techniques:
  - Neural networks
  - Decision tree learning
  - Support vector machines
  - Radial basis functions
  - Naive Bayes learning
  - k-nearest neighbor
  - ...

Let us look at supervised learning and some of these issues. First, there are two types of supervised learning. The first is regression. Given an input  $x$ , we want to find the value  $y$  for a function  $f$ . Usually, this is a continuous function.

The second task is classification. In this case, we have discrete classes either for face recognition,

handwritten character recognition, or credit risk assessment. Here we might have a numerical value, discrete, 0 to 10. On a 0 to 10 scale, what is the risk of granting this credit card to a certain person, and so on?

There are many techniques for supervised learning, including neural networks, which form the basis for deep learning, decision tree learning, support vector machines, radial basis functions, naive Bayes learning,  $k$ -nearest neighbor, and so on.

---

## Supervised Learning Issues

- How well will it do on training inputs?
- How well will it do on novel inputs?
  - Generalization.
- How many samples needed for sufficient performance and generalization?
  - Sample complexity
  - Curse of dimensionality<sub>1</sub>
  - Computational learning theory
- Catastrophic forgetting (online learning hard).
- What model to choose? (Inductive bias).

There are several issues to consider for supervised learning. How well will a particular algorithm do on the training input?

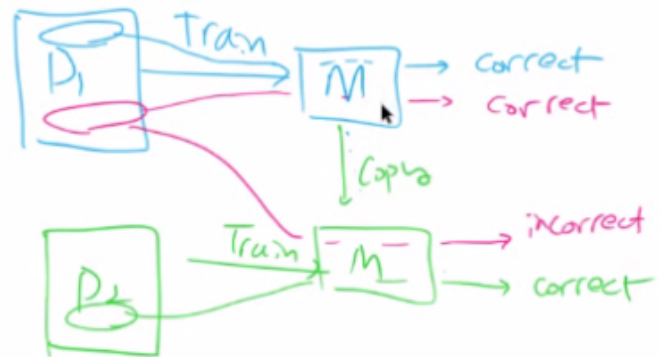
How well will it do on novel inputs? Novel inputs are inputs that are not included in the training inputs, and this leads to the concept of generalization. If our algorithm does very well on unseen inputs, then we say that it generalizes very well.

An important question in supervised learning is how many samples we need for sufficient performance and generalization. This is called sample complexity, and there are several concepts, like the curse of dimensionality, that are relevant to answering this question. And finally, there is a dedicated sub-field of machine learning called computation learning theory that deals with these topics.

Another important issue in supervised learning is catastrophic forgetting.

## Supervised Learning Issues

- How well will it do on training inputs?
- How well will it do on novel inputs?
  - Generalization.
- How many samples needed for sufficient performance and generalization?
  - Sample complexity
  - Curse of dimensionality
  - Computational learning theory
- Catastrophic forgetting (online learning hard).
- What model to choose? (Inductive bias).



Consider this scenario. We take Dataset 1 and train a model, so we have a trained model, and then we copy it over, and we train it with a new dataset, Dataset 2.

When we take input from Dataset 1 and put it through the training model, it will be correct.

However, when we take input from Dataset 1 and then put it through this model  $M$ , which was trained further on Dataset 2, then most likely, it will get incorrect. Whereas if we take input from Dataset 2, it passes through this training model, then it will be correct.

Of course, if we take the same input that we use here to get an incorrect answer from the second model, if we put it through the initial model where it was trained based on Dataset 1, it will be correct because that is what this model was trained on. Catastrophic forgetting means when we train a model with a new dataset, then catastrophic forgetting, of course, for the previous dataset that we used, which was learned by the same model, but it was overwritten by the new dataset.

# Supervised Learning Issues

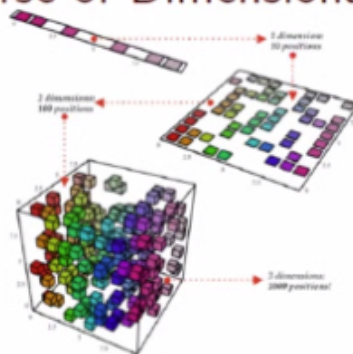
- How well will it do on training inputs?
- How well will it do on novel inputs?
  - Generalization.
- How many samples needed for sufficient performance and generalization?
  - Sample complexity
  - Curse of dimensionality<sub>1</sub>
  - Computational learning theory
- Catastrophic forgetting (online learning hard).
- What model to choose? (Inductive bias).

The last item here is what model to choose. So, given a dataset, should we use a neural network? Should we use a decision tree? Should we use the  $k$ -nearest neighbor? Or even using the same neural network, what type of neural network do we prefer? Do we prefer a deep and very complex model, or do we prefer a small and compact model?

This is called the inductive bias, and I will talk about this inductive bias in more detail later on.

---

## Curse of Dimensionality



From: Yoshua Bengio's page

- If you want 10 different values for each feature dimension, need exponentially many samples:
  - 1D: need  $10^1 = 10$
  - 2D: need  $10^2 = 100$
  - 3D: need  $10^3 = 1,000$

An important concept in computational learning theory relating to supervised learning is the concept of the curse of dimensionality. Let us consider this input. It is a 1-dimensional input, and let us say we want this coverage to include six samples within this input range. When we go to 2-D, if we have just six samples here, then there will be a lot of empty space in this 2-dimensional space. I just picked six random locations in this 2-dimensional space, and we can see there is a lot of empty space.

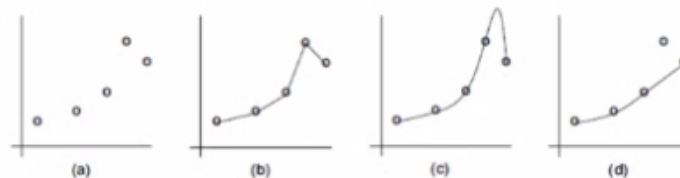
To get about the same coverage as in the 1-D case, we need six squared.

Likewise, once we move to 3-D, we need six cubed samples to get the same kind of coverage.

Another way of looking at this is if we want 10 different values for each feature dimension, we need exponentially many samples. For example, for 1-D, we need 10 to the 1, which is 10 samples. For 2-D, we need 10 to the 2. That is a hundred samples. For 3-D input space, we need 10 to the 3. That is 1000 inputs, and so on.

Now, try applying that to that 2,500-dimensional input image. We need 10 to the power of 2,500, which is a prohibitively large number.

## Inductive Bias

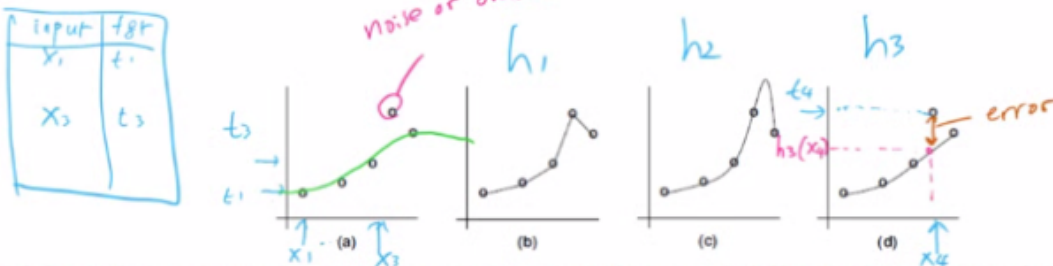


Given (a) as the training data, we can come up with several different hypotheses (model): (b) to (d)

- selection of one hypothesis (model) over another is called an **inductive bias**.
  - exact match to training data
  - prefer imprecise but smooth approximation
  - etc.

Another important concept in supervised learning is inductive bias. Let us say we are given this dataset.

## Inductive Bias



Given (a) as the training data, we can come up with several different hypotheses (model): (b) to (d)

- selection of one hypothesis (model) over another is called an **inductive bias**.
  - exact match to training data
  - prefer imprecise but smooth approximation
  - etc.

Each data point here represents one row in the dataset. We sample  $x_1$ . It maps to  $t_1$ . So, we have the input  $x_1$  and target value of  $t_1$ . So, that is the first sample, and so on. And let us say this is the third input. Input is  $x_3$ . Then the target value is  $t_3$ . That is another row. So, in all, we have five rows in the dataset.

There are several different models to use to fit this data. We can do it through linear interpolation. We can have a higher-order polynomial like that or a lower-order polynomial like that to fit the data. In machine learning theory, these are also called hypotheses, and in fact, these are three different types of hypotheses.

Let us call them  $h_1$ ,  $h_2$ , and  $h_3$ .

Given this choice, which hypothesis would we choose? If we care about exact matching with the training data set, we will probably go with  $h_1$  and or  $h_2$ . If we prefer a simple model with only a few parameters, then we will probably go with  $h_3$ . But the downside is that there will be an error here on the training set.

When we plug in  $x_4$  into  $h_3$ , then this would be the output value, whereas the true value in the training set for  $x_4$  is  $t_4$ . That would be this value. So, this amount here is the error made by our hypothesis  $h_3$ .

Whereas that kind of error does not occur for  $h_1$  and  $h_2$ .

But what if the true function is this green function, and this data point in our training set is either a noise or an outlier, meaning that there is some kind of error in constructing this data set? Then  $h_3$  suddenly looks better than  $h_1$  and  $h_2$ .

But the problem is we never know whether this is the case or not. So, we are faced with the question of choosing between these three or even more choices. If we prefer a simple and small

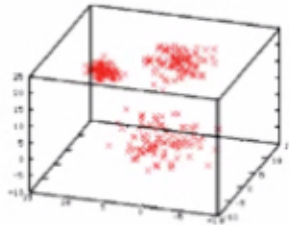
number of parameters in the model, then we will go with this and end up actually dealing with this kind of situation better.

But again, we do not know whether this data point here is actually a good one or a bad one.

So, given a dataset like that, choosing what type of hypotheses to use is called inductive bias.

---

## Unsupervised Learning



Clustering, feature extraction, blind source separation, dimensionality reduction, etc.

- Principal Component Analysis (PCA)
- Self-Organizing Maps (SOM)
- Independent Component Analysis (ICA)
- Multi-Dimensional Scaling (MDS)
- ISOMAP, Locally Linear Embedding (LLE)
- t-distr. Stochastic Neighbor Embedding (t-SNE)

I

Let us move on to unsupervised learning. Again, this is a typical unsupervised learning dataset in the simplest form. We use unsupervised learning for many different tasks, such as clustering, feature extraction, line source separation dimensionality reduction, and so on.

These are some of the techniques that are commonly used. Principal Component Analysis, otherwise known as PCA, Self-Organizing Maps (SOM), Independent Component Analysis (ICA), multi-dimensional scaling (MDS), ISOMAP, Locally Linear Embedding, and this is generally abbreviated as LLE, and t-distributed stochastic neighbor embedding or t-SNE.

In this course, we will look at Self-Organizing Maps in more detail.

---

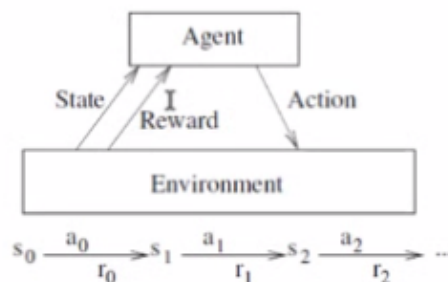
# Unsupervised Learning Issues

- Discovering structure.
- Discovering features.
- Removing redundancy.
- How many clusters?  $k$
- What distance measure to use?

What are the questions or issues regarding unsupervised learning? In unsupervised learning, we want to discover structure, discover features, and remove redundancy in the data. We want to ask questions like how many clusters there are and what distance measure to use.

---

## Reinforcement Learning



- Very different from supervised and unsupervised learning.
- Multi agent control, robot control, game playing, scheduling, etc.
- Techniques:
  - Value function-based: Q-learning, Temporal difference (TD) learning
  - Direct policy search: Neuroevolution, genetic algorithms, policy gradient

Next is reinforcement learning. Reinforcement learning is very different from supervised and unsupervised learning. We have an agent who perceives the state and takes action. As a result, the environment will transition the agent into a new state and give some kind of reward or penalty. Based on the updated state and the reward information, the agent makes another action, and the environment will transition the agent into the next state and, again, provide the reward. The agent

has to take a series of actions to maximize the reward that is gained over the course of time.

Reinforcement learning is applied to multi-agent control, robot control, game playing, scheduling, autonomous driving, etc. There are two basic types of techniques, value function-based and direct policy search.

Value-function-based approaches include Q-learning and temporal difference learning and its variance.

Here we want to evaluate the value of a particular state or evaluate the value of taking a particular action in a particular state. That is called the Q function.

The second type is direct policy search. Here we want to find the mapping between the state to the action directly, and these include neuroevolution, genetic algorithms, and policy gradient.

---

## Reinforcement Learning Issues

- Discrete states and actions were a norm.
- Scalability an issue.
- Unrealistic assumptions for theoretical guarantees: state-action pair visited infinitely often.
- Online learning, safety, transfer, imitation, etc.
- Deep reinforcement learning disrupted a lot of the traditional assumptions and limitations.

There are several things to consider in reinforcement learning. First of all, generally, the states and actions are discrete.

Due to this, there was a huge scalability issue. Also, there are nice theorems in traditional reinforcement learning theory, but they make unrealistic assumptions to make certain theoretical guarantees. For example, it can be proven that the learned outcome is optimal when the state-action pair is visited infinitely often. And there are many interesting challenges like online learning, learning as we go, safety transfer learning, imitation learning, and so on.

So, many of these have been huge issues in traditional reinforcement learning, but deep reinforcement learning disrupted many of these traditional assumptions and limitations, and they are making huge progress.

---

# Summary (ML Intro)

- Machine learning is a rapidly developing field with great promise:
  - Big data
  - Deep neural networks
  - Fast computing: GPGPU, cloud, etc.
- Three types of ML:
  - Supervised learning
  - Unsupervised learning
  - Reinforcement learning
- Need to look beyond ML:
  - ML good at solving problems, but not posing problems (Choe and Mann 2012).

In summary, machine learning is a rapidly developing field with great promise, and this was largely fueled by these three things, huge amounts of data, very complex deep neural networks, and fast computing to support all of these. For example, GPGPU, which stands for General Purpose Computing on the GPU, and many of these cloud services accommodate the computing needs.

There are three types of machine learning, supervised, unsupervised, and reinforcement learning, and eventually, we need to look beyond machine learning. Machine learning is good at solving problems, but it is not good at posing these problems. If we give a task to machine learning, then it will solve and give us an answer, whereas it will not actually come up with its own question or own dataset or its own task.

In the following series of lectures, we look at representative machine-learning algorithms and their applications.